

**PRACTICE EXERCISES OF THE MICROPROCESSORS
& MICROCONTROLLERS**

Instructor: The Tung Than

Student's name: Gia Hung Nguyen

Student code: 24520603

**PRACTICE REPORT NO 5
ADDITION OF TWO 32-BIT NUMBERS
ON THE 8086 PROCESSOR**

- I. Chương trình mẫu**
- II. Lưu đồ giải thuật cộng hai số 32-bit**
- III. Xây dựng chương trình Assembly 8086**
- IV. Kết quả mô phỏng**

I. Chương trình mẫu

Chương trình mẫu add-sub thực hiện phép cộng và phép trừ đơn giản trên dữ liệu 8-bit của vi xử lý 8086. Sau khi tính toán xong, chương trình in kết quả ra màn hình dưới dạng nhị phân.

1. Chương trình mẫu

```
1 name "add-sub"
2
3 org 100h
4
5 mov al, 5      ; bin=00000101b
6 mov bl, 10     ; hex=0ah or bin=00001010b
7
8 ; 5 + 10 = 15 (decimal) or hex=0fh or bin=00001111b
9 add bl, al
10
11 ; 15 - 1 = 14 (decimal) or hex=0eh or bin=00001110b
12 sub bl, 1
13
14 ; print result in binary:
15 mov cx, 8
16 print: mov ah, 2 ; print function.
17         mov dl, '0'
18         test bl, 10000000b ; test first bit.
19         jz zero
20         mov dl, '1'
21 zero:   int 21h
22         shl bl, 1
23 loop print
24
25 ; print binary suffix:
26 mov dl, 'b'
27 int 21h
28
29 ; wait for any key press:
30 mov ah, 0
31 int 16h
32
33 ret
```

2. Giải thích chương trình mẫu

Ban đầu, chương trình gán giá trị 5 vào thanh ghi AL và giá trị 10 vào thanh ghi BL:

$$AL = 5 = 00000101_2$$

$$BL = 10 = 00001010_2$$

Sau đó, lệnh:

```
1 add bl, al
```

thực hiện phép cộng giữa BL và AL:

$$BL = BL + AL = 10 + 5 = 15$$

Kết quả sau phép cộng là:

$$BL = 15 = 00001111_2 = 0F_{16}$$

Tiếp theo, lệnh:

```
1 sub bl, 1
```

thực hiện phép trừ:

$$BL = BL - 1 = 15 - 1 = 14$$

Vì vậy, kết quả cuối cùng được lưu trong thanh ghi BL là:

$$BL = 14 = 00001110_2 = 0E_{16}$$

Sau khi tính toán, chương trình in giá trị trong BL ra màn hình dưới dạng nhị phân. Thanh ghi CX được gán bằng 8 vì chương trình cần in đủ 8 bit.

```
1 mov cx, 8
```

Trong mỗi vòng lặp, lệnh:

```
1 test bl, 10000000b
```

được dùng để kiểm tra bit cao nhất của thanh ghi BL. Nếu bit cao nhất bằng 0, chương trình in ký tự '0'. Nếu bit cao nhất bằng 1, chương trình in ký tự '1'.

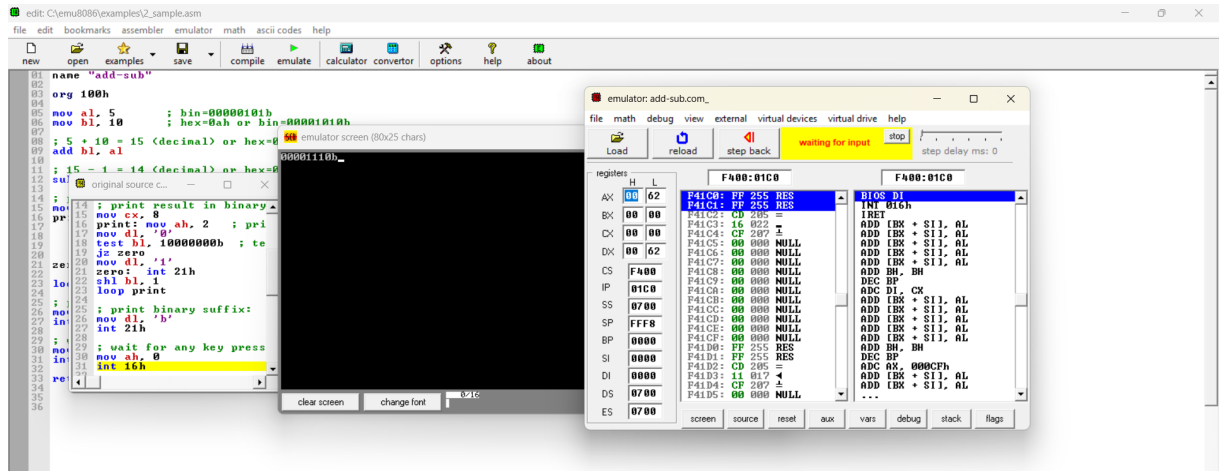
Sau khi in xong một bit, lệnh:

```
1 shl bl, 1
```

dịch trái thanh ghi BL một bit. Nhờ đó, bit kế tiếp sẽ được đưa lên vị trí cao nhất để kiểm tra ở vòng lặp tiếp theo.

Sau 8 lần lặp, chương trình in thêm ký tự 'b' để biểu diễn kết quả ở dạng nhị phân. Cuối cùng, chương trình chờ người dùng nhấn phím bất kỳ rồi kết thúc.

Kết quả hiển thị của chương trình mẫu là: **00001110b**



Hình 1. Kết quả hiển thị chương trình mẫu

II. Lưu đồ giải thuật cộng hai số 32-bit

1. Lưu đồ giải thuật

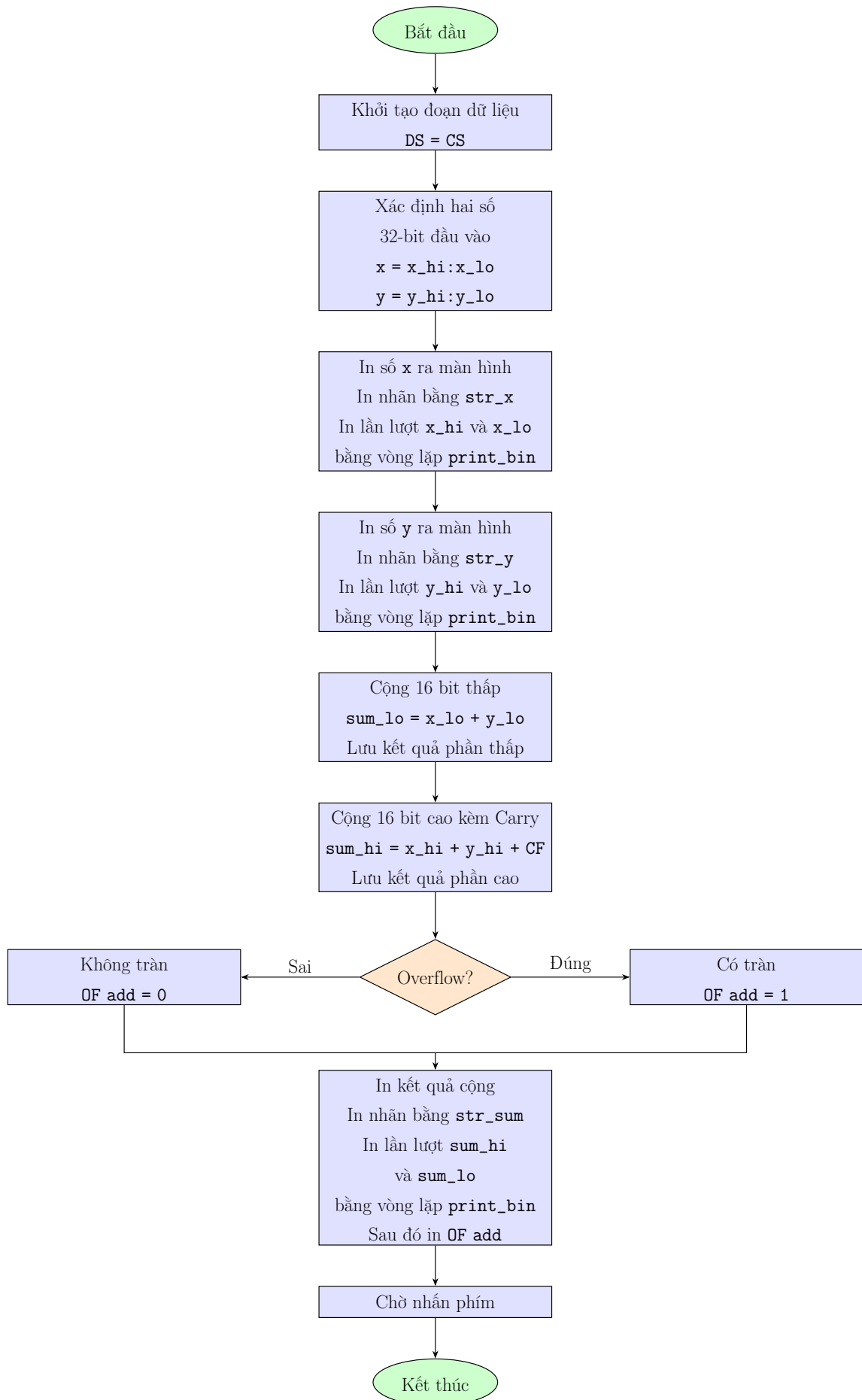
Vi xử lý 8086 là vi xử lý 16-bit, do đó không thể xử lý trực tiếp một số 32-bit chỉ bằng một thanh ghi duy nhất. Vì vậy, trong chương trình, mỗi số 32-bit được chia thành hai phần 16-bit:

$$x = x_{HIGH} : x_{LOW}$$

$$y = y_{HIGH} : y_{LOW}$$

Trong đó:

- **HIGH**: 16 bit cao.
- **LOW**: 16 bit thấp.



Hình 2. Lưu đồ giải thuật cộng hai số 32-bit trên vi xử lý 8086

2. Giải thích lưu đồ

Đầu tiên, chương trình khởi tạo đoạn dữ liệu để truy cập đúng các biến đã khai báo. Sau đó, hai số đầu vào được hiển thị ra màn hình dưới dạng nhị phân 32-bit. Việc in được thực hiện theo thứ tự 16 bit cao trước, 16 bit thấp sau. Mỗi word 16-bit được in bằng vòng lặp `print_bin`; vòng lặp này chạy 16 lần, mỗi lần kiểm tra một bit, in ra 0 hoặc 1, rồi dịch trái để xét bit tiếp theo.

Tiếp theo, chương trình cộng 16 bit thấp của hai số và lưu vào phần thấp của kết quả. Nếu phép cộng này phát sinh nhớ, cờ Carry sẽ được dùng cho bước cộng phần cao. Sau đó, chương trình cộng 16 bit cao kèm theo Carry để thu được kết quả cộng 32-bit chính xác.

Sau khi cộng xong, chương trình kiểm tra cờ Overflow. Nếu không tràn số có dấu thì `OF add = 0`, ngược lại `OF add = 1`. Cuối cùng, chương trình hiển thị kết quả cộng dưới dạng nhị phân 32-bit, hiển thị cờ Overflow, chờ người dùng nhấn phím rồi kết thúc.

III. Xây dựng chương trình Assembly

1. Giải thích chương trình

ASSEMBLY	GIẢI THÍCH
<code>name "add-sub-32bit"</code> <code>org 100h</code> <code>jmp start</code>	Khai báo chương trình: Lệnh <code>ORG 100H</code> cho biết chương trình được viết theo dạng <code>.COM</code> , bắt đầu thực thi tại địa chỉ offset 100H.

ASSEMBLY	GIẢI THÍCH
<pre> x_hi dw XXXXh x_lo dw XXXXh y_hi dw XXXXh y_lo dw XXXXh </pre>	Khai báo hai số 32-bit: Do 8086 là vi xử lý 16-bit nên không thể lưu trực tiếp một số 32-bit trong một thanh ghi duy nhất. Vì vậy, mỗi số 32-bit được chia thành hai word 16-bit. Trong chương trình, biến <i>x</i> được dùng để biểu diễn số thứ nhất, còn biến <i>y</i> được dùng để biểu diễn số thứ hai. Số thứ nhất: $x = XXXXXXXXH$ được tách thành: $x_{hi} = XXXXH, \quad x_{lo} = XXXXH$ Số thứ hai: $y = XXXXXXXXH$ được tách thành: $y_{hi} = XXXXH, \quad y_{lo} = XXXXH$
<pre> sum_hi dw ? sum_lo dw ? diff_hi dw ? diff_lo dw ? </pre>	Khai báo biến lưu kết quả: Các biến <i>sum_hi</i> và <i>sum_lo</i> dùng để lưu kết quả của phép cộng 32-bit. Trong đó, <i>sum_hi</i> lưu 16 bit cao và <i>sum_lo</i> lưu 16 bit thấp. Tương tự, <i>diff_hi</i> và <i>diff_lo</i> dùng để lưu kết quả của phép trừ 32-bit.

ASSEMBLY	GIẢI THÍCH
<pre>add_of db '0' sub_of db '0'</pre>	Khai báo biến lưu cờ Overflow: Biến <code>add_of</code> dùng để lưu trạng thái cờ Overflow sau phép cộng. Biến <code>sub_of</code> dùng để lưu trạng thái cờ Overflow sau phép trừ. Hai biến này được lưu trực tiếp dưới dạng ký tự '0' hoặc '1' để thuận tiện khi in ra màn hình. Nếu <code>OF = 0</code>, phép toán không bị tràn. Nếu <code>OF = 1</code>, phép toán bị tràn.
<pre>str_x db 'x = \$' str_y db 'y = \$' str_sum db 'sum = \$' str_diff db 'diff = \$' str_add_of db 'OF add = \$' str_sub_of db 'OF sub = \$' str_bnl db 'b', 13, 10, '\$' newline db 13, 10, '\$'</pre>	Khai báo chuỗi hiển thị: Các chuỗi này dùng để in nhãn kết quả ra màn hình, ví dụ: <code>x =</code>, <code>y =</code>, <code>sum =</code>, <code>diff =</code>, <code>OF add =</code>, <code>OF sub =</code>. Chuỗi <code>str_bnl</code> dùng để in ký tự <code>b</code> sau chuỗi nhị phân, sau đó xuống dòng bằng hai mã ASCII 13, 10. Mỗi chuỗi kết thúc bằng ký tự <code>\$</code> vì chương trình sử dụng ngắt INT 21H với hàm AH = 09H để in chuỗi.
<pre>start: mov ax, cs mov ds, ax</pre>	Khởi tạo thanh ghi đoạn dữ liệu: Chương trình gán giá trị của thanh ghi CS cho DS để các lệnh truy cập biến dữ liệu như <code>x_hi</code>, <code>x_lo</code>, <code>y_hi</code>, <code>y_lo</code> hoạt động đúng.
<pre>lea dx, str_x call print_str mov bx, [x_hi] call print_bin mov bx, [x_lo] call print_bin lea dx, str_bnl call print_str</pre>	In số thứ nhất x: Đầu tiên, chương trình in chuỗi <code>str_x</code>. Sau đó, chương trình lần lượt nạp <code>x_hi</code> và <code>x_lo</code> vào thanh ghi BX, rồi gọi nhãn <code>print_bin</code>. Mỗi lần gọi <code>print_bin</code>, chương trình in ra 16 bit nhị phân. Vì <code>x_hi</code> được in trước và <code>x_lo</code> được in sau, kết quả cuối cùng là số x đầy đủ 32 bit.

ASSEMBLY	GIẢI THÍCH
<pre>lea dx, str_y call print_str mov bx, [y_hi] call print_bin mov bx, [y_lo] call print_bin lea dx, str_bnl call print_str</pre>	In số thứ hai y: Tương tự như khi in x, chương trình in chuỗi <code>str_y</code>, sau đó in lần lượt <code>y_hi</code> và <code>y_lo</code>.
<pre>mov ax, [x_lo] add ax, [y_lo] mov [sum_lo], ax</pre>	Cộng 16 bit thấp: Chương trình nạp word thấp của x vào thanh ghi AX, sau đó cộng với word thấp của y. Kết quả của phép cộng 16 bit thấp được lưu vào biến <code>sum_lo</code>. Nếu phép cộng này vượt quá 16 bit, cờ Carry CF sẽ được bật để chuyển phần nhớ sang phép cộng word cao.
<pre>mov ax, [x_hi] adc ax, [y_hi] mov [sum_hi], ax</pre>	Cộng 16 bit cao kèm Carry: Sau khi cộng word thấp, chương trình tiếp tục cộng hai word cao bằng lệnh ADC. Lệnh ADC thực hiện phép cộng có kèm cờ Carry: $sum_{hi} = x_{hi} + y_{hi} + CF$ Nhờ vậy, nếu phép cộng ở 16 bit thấp có phát sinh nhớ, phần nhớ này sẽ được cộng tiếp vào 16 bit cao. Kết quả được lưu vào biến <code>sum_hi</code>.
<pre>mov byte ptr [add_of], '0' jno no_add_overflow mov byte ptr [add_of], '1'</pre>	Kiểm tra Overflow sau phép cộng: Sau phép cộng word cao, chương trình kiểm tra cờ Overflow OF. Ban đầu, biến <code>add_of</code> được gán bằng '0'. Nếu lệnh JNO cho biết không có tràn, chương trình giữ nguyên giá trị này. Nếu có tràn số có dấu, chương trình gán <code>add_of = '1'</code>. Cờ OF được dùng để phát hiện tràn trong phép toán số có dấu.

ASSEMBLY	GIẢI THÍCH
<pre>lea dx, str_sum call print_str mov bx, [sum_hi] call print_bin mov bx, [sum_lo] call print_bin lea dx, str_bnl call print_str</pre>	In kết quả cộng: Chương trình in chuỗi <code>str_sum</code>, sau đó in lần lượt <code>sum_hi</code> và <code>sum_lo</code> bằng nhãn <code>print_bin</code>. Sau khi in đủ 32 bit của kết quả cộng, chương trình in thêm ký tự <code>b</code> để biểu diễn rằng kết quả đang ở dạng nhị phân.
<pre>lea dx, str_add_of call print_str mov dl, [add_of] call print_char lea dx, newline call print_str</pre>	In cờ Overflow của phép cộng: Chương trình in chuỗi <code>OF add =</code>, sau đó đưa giá trị của biến <code>add_of</code> vào thanh ghi DL và gọi <code>print_char</code> để in ra màn hình. Giá trị được in là 0 nếu không tràn và 1 nếu có tràn số có dấu.
<pre>mov ax, [x_lo] sub ax, [y_lo] mov [diff_lo], ax</pre>	Trừ 16 bit thấp: Chương trình nạp word thấp của <code>x</code> vào thanh ghi AX, sau đó trừ word thấp của <code>y</code>. Kết quả của phép trừ 16 bit thấp được lưu vào biến <code>diff_lo</code>. Nếu phép trừ này cần mượn, cờ Carry CF sẽ được bật để biểu diễn bit mượn cho phép trừ word cao.
<pre>mov ax, [x_hi] sbb ax, [y_hi] mov [diff_hi], ax</pre>	Trừ 16 bit cao kèm Borrow: Sau khi trừ word thấp, chương trình tiếp tục trừ hai word cao bằng lệnh <code>SBB</code>. Lệnh <code>SBB</code> thực hiện phép trừ có kèm bit mượn: $diff_{hi} = x_{hi} - y_{hi} - CF$ Trong 8086, bit mượn của phép trừ được biểu diễn thông qua cờ Carry CF. Do đó, nếu phép trừ word thấp có phát sinh mượn, lệnh <code>SBB</code> sẽ trừ thêm phần mượn này ở word cao. Kết quả được lưu vào biến <code>diff_hi</code>.

ASSEMBLY	GIẢI THÍCH
<pre> mov byte ptr [sub_of], '0' jno no_sub_overflow mov byte ptr [sub_of], '1' </pre>	Kiểm tra Overflow sau phép trừ: Sau phép trừ word cao, chương trình kiểm tra cờ Overflow OF. Ban đầu, biến <code>sub_of</code> được gán bằng '0'. Nếu không có tràn, chương trình giữ nguyên giá trị này. Nếu có tràn số có dấu, chương trình gán <code>sub_of</code> = '1'.
<pre> lea dx, str_diff call print_str mov bx, [diff_hi] call print_bin mov bx, [diff_lo] call print_bin lea dx, str_bnl call print_str </pre>	In kết quả trừ: Chương trình in chuỗi <code>str_diff</code>, sau đó in lần lượt <code>diff_hi</code> và <code>diff_lo</code>.
<pre> lea dx, str_sub_of call print_str mov dl, [sub_of] call print_char lea dx, newline call print_str </pre>	In cờ Overflow của phép trừ: Chương trình in chuỗi OF <code>sub</code> =, sau đó in giá trị của biến <code>sub_of</code>. Giá trị này là 0 nếu phép trừ không bị tràn và là 1 nếu phép trừ bị tràn số có dấu.
<pre> print_str: mov ah, 09h int 21h ret </pre>	Nhãn in chuỗi: Nhãn <code>print_str</code> sử dụng ngắt INT 21H với hàm AH = 09H. Địa chỉ chuỗi cần in được đặt trong thanh ghi DX. Chuỗi phải kết thúc bằng ký tự \$.
<pre> print_char: mov ah, 02h int 21h ret </pre>	Nhãn in ký tự: Nhãn <code>print_char</code> sử dụng ngắt INT 21H với hàm AH = 02H. Ký tự cần in được đặt trong thanh ghi DL. Nhãn này được dùng để in từng bit 0/1 và in giá trị của cờ Overflow.

ASSEMBLY	GIẢI THÍCH
<pre> print_bin: mov cx, 16 next_bit: mov dl, '0' test bx, 1000000000000000b jz bit_zero mov dl, '1' bit_zero: call print_char shl bx, 1 loop next_bit ret </pre>	Nhân in một word nhị phân: Nhân <code>print_bin</code> in giá trị trong thanh ghi <code>BX</code> thành chuỗi 16 bit nhị phân. Đầu tiên, chương trình gán <code>CX = 16</code> để thực hiện vòng lặp 16 lần. Ở mỗi vòng lặp, chương trình mặc định ký tự cần in là <code>'0'</code>, sau đó dùng lệnh <code>TEST BX, 1000000000000000B</code> để kiểm tra bit cao nhất của <code>BX</code>. Nếu bit cao nhất bằng 1, chương trình đổi ký tự cần in thành <code>'1'</code>. Sau khi in xong một bit, lệnh <code>SHL BX, 1</code> dịch trái <code>BX</code> một bit để đưa bit kế tiếp lên vị trí cao nhất. Vòng lặp tiếp tục cho đến khi in đủ 16 bit.
<pre> mov ah, 0 int 16h ret </pre>	Kết thúc chương trình: Sau khi in tất cả kết quả, chương trình dùng ngắt <code>INT 16H</code> với <code>AH = 0</code> để chờ người dùng nhấn phím bất kỳ. Sau đó, lệnh <code>RET</code> được thực thi để kết thúc chương trình.

IV. Kết quả mô phỏng

1. Ví dụ 1: $x = 15, y = 5$

Khởi tạo giá trị `x, y`:

```

1 x_hi dw 0000h
2 x_lo dw 000Fh
3
4 y_hi dw 0000h
5 y_lo dw 0005h

```

Kết quả:

$$x + y = 15 + 5 = 20$$

$$x - y = 15 - 5 = 10$$

Kết quả hiển thị mong đợi:

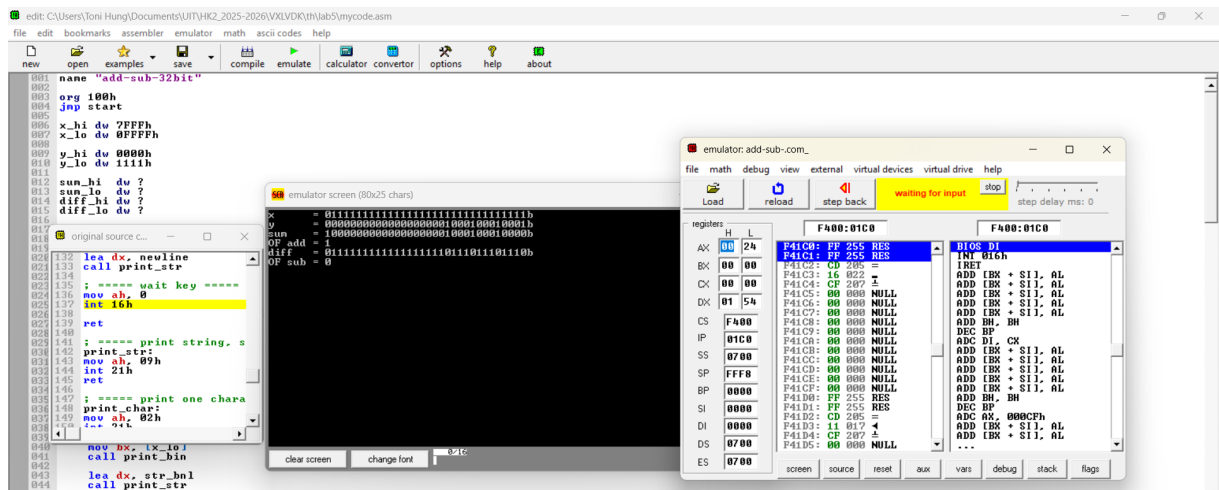
Nếu xét theo số có dấu 32-bit, 7FFFFFFFH là số dương lớn nhất. Khi cộng thêm một số dương, kết quả chuyển sang bit dấu bằng 1 nên xảy ra tràn số có dấu.

Kết quả phép trừ:

$$7FFFFFFFH - 00001111H = 7FFFEEEH$$

Kết quả hiển thị mong đợi:

```
x = 01111111111111111111111111111111b
y = 0000000000000000000000001000100010001b
sum = 1000000000000000000000001000100010000b
OF add = 1
diff = 0111111111111111111111101110111011101110b
OF sub = 0
```



Hình 5. Kết quả mô phỏng ví dụ 3

4. Ví dụ 4: Tràn khi trừ

Để tạo tràn số có dấu khi trừ, chọn:

$$x = 7FFFFFFFH$$

$$y = FFFFFFFFH$$

Trong biểu diễn bù 2 32-bit, FFFFFFFFH tương ứng với giá trị -1 .

Khởi tạo giá trị x, y :

```

1 x_hi dw 7FFFh
2 x_lo dw 0FFFFh
3
4 y_hi dw 0FFFFh
5 y_lo dw 0FFFFh

```

Khi trừ:

$$7FFFFFFFH - FFFFFFFFH = 80000000H$$

Tương đương:

$$2147483647 - (-1) = 2147483648$$

Giá trị này vượt quá miền biểu diễn của số có dấu 32-bit, nên cờ Overflow được bật.

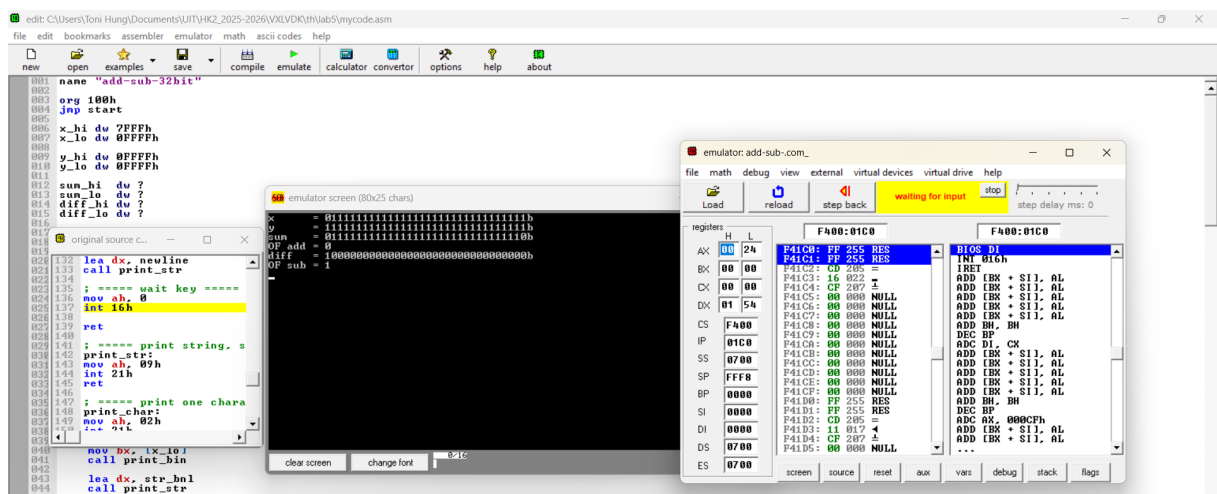
Kết quả hiển thị mong đợi:

```

x = 01111111111111111111111111111111b
y = 11111111111111111111111111111111b
sum = 01111111111111111111111111111110b
OF add = 0

diff = 10000000000000000000000000000000b
OF sub = 1

```



Hình 6. Kết quả mô phỏng ví dụ 4

5. Link video mô phỏng

Link video mô phỏng quá trình biên dịch, chạy chương trình và quan sát kết quả trên emu8086:

[https://drive.google.com/drive/folders/1gPrxuJi0as6abKVt-srvJol7H-ktwH_5?
usp=sharing](https://drive.google.com/drive/folders/1gPrxuJi0as6abKVt-srvJol7H-ktwH_5?usp=sharing)